

Εθνικό Μετσόβιο Πολυτεχνείο



Εργαστήριο Μικροϋπολογιστών

7η Εργαστηριακή Άσκηση

Χριστόφορος Χαραλάμπους 03121614

Φίλιππος Γιαννακόπουλος 03121629

Ομάδα 17

Ζήτημα 7

Το Ζήτημα 7 βρίσκεται στο αρχείο erg7.2.c.

Το διάβασμα του θερμομέτρου γίνεται βάση της διαδικασίας που περιγράφηκε στην οδηγία. Ακολούθως, η μετατροπή από προσημασμένο αριθμό έγινε κάνοντας cast την μεταβλητή που διαβάσαμε από uint16_t σε int16_t. Και μετά, γίνεται μετατροπή σε double για πιο εύκολη διαχείριση των δεκαδικών ψηφίων.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define PCA9555_0_ADDRESS 0x40 // A0=A1=A2=0 by hardware
#define TWI_READ 1 // reading from twi device
#define TWI_WRITE 0 // writing to twi device
#define SCL_CLOCK 100000L // twi clock in Hz

//FscL=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START 0x08
#define TW_REP_START 0x10
//----- Master Transmitter -----
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28
//----- Master Receiver -----
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b1111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

uint8_t PREVIOUS = 0;

// Initializes TWI clock
void twi_init() {
    TWSR0 = 0; // Prescaler = 1
    TWBR0 = TWBR0_VALUE; // SCL clock 100kHz
}

// Read one byte from twi device (while requesting more)
unsigned char twi_readAck() {
    TWCRR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while (!(TWCRR0 & (1<<TWINT)));

    return TWDRO;
```

```

}

// Read one byte from twi device (followed by a stop condition)
unsigned char twi_readNak() {
    TWCR0 = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR0 & (1<<TWINT)));

    return TWDRO;
}

// Sends start condition, address and transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_start(unsigned char address) {
    uint8_t twi_status;

    // Send START
    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        return 1;

    // Send device address
    TWDRO = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received.
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK))
        return 1;

    return 0;
}

// Sends start condition, address and transfer direction
// Waits until device is ready
void twi_start_wait(unsigned char address) {
    uint8_t twi_status;

    while (1) {
        // Send START
        TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

        // Wait till START transmission is completed
        while (!(TWCR0 & (1<<TWINT)));

        // Check TWI Status
        twi_status = TW_STATUS & 0xF8;
        if ((twi_status != TW_START) && (twi_status != TW_REP_START))
            continue;

        // Send device address
        TWDRO = address;
        TWCR0 = (1<<TWINT) | (1<<TWEN);
    }
}

```

```

// Wait till address transmission is completed and ACK/NACK has been received
while (!(TWCRO & (1<<TWINT)));

// Check TWI Status
twi_status = TW_STATUS & 0xF8;
if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK)) {
    // Device is busy, send STOP condition
    TWCRO = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCRO & (1<<TWSTO)));

    continue;
}

break;
}

// Send one byte to the TWI device
// Returns 0 for write success, 1 for write failure
unsigned char twi_write(unsigned char data) {
    // Send data to the (already addressed) device
    TWDR0 = data;
    TWCRO = (1<<TWINT) | (1<<TWEN);

    // Wait till transmission finishes
    while (!(TWCRO & (1<<TWINT)));

    if ((TW_STATUS & 0xF8) != TW_MT_DATA_ACK)
        return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_rep_start(unsigned char address) {
    return twi_start(address);
}

// Stops data transfer and releases TWI bus
void twi_stop() {
    // Send STOP condition
    TWCRO = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCRO & (1<<TWSTO)));
}

// PCA9555 functions

void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value) {
    PREVIOUS = value;
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg) {

```

```

uint8_t ret_val;

twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
twi_write(reg);
twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
ret_val = twi_readNak();
twi_stop();

return ret_val;
}

// Returns 1 if device is detected, 0 if no.
uint8_t one_wire_reset() {
    DDRD |= 1<<4;
    PORTD = 0;

    _delay_us(480);

    DDRD = 0;
    PORTD = 0;

    _delay_us(100);

    uint8_t temp = PIND;

    _delay_us(380);

    return (temp & 1<<4) ? 0 : 1;
}

uint8_t one_wire_receive_bit() {
    DDRD |= 1<<4;
    PORTD = 0;

    _delay_us(2);

    DDRD = 0;
    PORTD = 0;

    _delay_us(10);

    uint8_t temp = (PIND & 1<<4) ? 1 : 0;

    _delay_us(49);

    return temp;
}

void one_wire_transmit_bit(uint8_t bit) {
    DDRD |= 1<<4;
    PORTD = 0;

    _delay_us(2);

    PORTD = bit<<4;

    _delay_us(58);

    DDRD = 0;
    PORTD = 0;
}

```

```

    _delay_us(1);
}

uint8_t one_wire_receive_byte() {
    uint8_t output = 0x00;

    for (uint8_t i = 0; i < 8; i++) {
        output = output>>1;
        if (one_wire_receive_bit()) {
            output |= 1<<7;
        }
    }

    return output;
}

void one_wire_transmit_byte(uint8_t byte) {
    uint8_t temp = 1;
    for (uint8_t i = 0; i < 8; i++) {
        if (byte & temp) {
            one_wire_transmit_bit(1);
        } else {
            one_wire_transmit_bit(0);
        }
        temp = temp << 1;
    }
}

uint16_t temp() {
    if (!one_wire_reset()) {
        return 0x8000;
    }

    one_wire_transmit_byte(0xCC);

    one_wire_transmit_byte(0x44);

    while (!one_wire_receive_bit());

    if (!one_wire_reset()) {
        return 0x8000;
    }

    one_wire_transmit_byte(0xCC);

    one_wire_transmit_byte(0xBE);

    uint16_t output = one_wire_receive_byte();
    output |= one_wire_receive_byte()<<8;

    return output;
}

// LCD Commands

void write2(unsigned char input) {
    unsigned char prev = PREVIOUS;

    unsigned char write = (input & 0xF0) | (prev & 0x0F);
    PCA9555_0_write(REG_OUTPUT_0, write);
}

```

```

    write |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, write);
    write &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, write);

    write = ((input & 0x0F) << 4) | (prev & 0x0F);
    PCA9555_0_write(REG_OUTPUT_0, write);

    write |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, write);
    write &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, write);
}

void lcd_data(unsigned char input) {
    unsigned char prev = PREVIOUS;
    prev |= (1<<2);

    PCA9555_0_write(REG_OUTPUT_0, prev);
    write2(input);
    _delay_us(250);
}

void lcd_command(unsigned char input) {
    unsigned char prev = PREVIOUS;
    prev &= 0b11111011;

    PCA9555_0_write(REG_OUTPUT_0, prev);
    write2(input);
    _delay_us(250);
}

void lcd_nextline() {
    lcd_command(0b11000000);
}

void lcd_clear() {
    lcd_command(0x01);
    _delay_ms(5);
}

void lcd_init() {
    _delay_ms(200);

    // Switch to 8bit mode
    unsigned char write = 0x30;
    PCA9555_0_write(REG_OUTPUT_0, write);

    unsigned char temp = write;

    temp |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, temp);
    temp &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, temp);
    _delay_us(250);

    temp = write;

    // Switch to 8bit mode
    PCA9555_0_write(REG_OUTPUT_0, write);

```

```

temp |= (1<<3);
PCA9555_0_write(REG_OUTPUT_0, temp);
temp &= 0b11110111;
PCA9555_0_write(REG_OUTPUT_0, temp);
_delay_us(250);

temp = write;

// Switch to 8bit mode
PCA9555_0_write(REG_OUTPUT_0, write);

temp |= (1<<3);
PCA9555_0_write(REG_OUTPUT_0, temp);
temp &= 0b11110111;
PCA9555_0_write(REG_OUTPUT_0, temp);
_delay_us(250);

// Switch to 8bit mode
write = 0x20;
temp = write;
PCA9555_0_write(REG_OUTPUT_0, write);

temp |= (1<<3);
PCA9555_0_write(REG_OUTPUT_0, temp);
temp &= 0b11110111;
PCA9555_0_write(REG_OUTPUT_0, temp);
_delay_us(250);

lcd_command(0x28);

lcd_command(0x0c);

lcd_clear();

lcd_command(0x06);
}

void display_number(uint8_t input) {
    lcd_data(0b00110000 | (input & 0x0F));
}

unsigned char thousands, hundreds, tens, units;

void binary_to_bcd(uint16_t input) {
    thousands = input / 1000;
    input = input % 1000;

    hundreds = input / 100;
    input = input % 100;

    tens = input / 10;
    units = input % 10;
}

const char no_device[] = "NO Device";

int main(void) {
    twi_init();

    // Configure EXT_PORT1 as output
    PCA9555_0_write(REG_CONFIGURATION_0, 0x00);

```

```

// Enable LCD
lcd_init();

uint8_t no_device_status = 0;

while(1) {
    uint16_t reading = temp();
    if (reading == 0x8000) {
        if (no_device_status) continue;
        lcd_clear();
        for (uint8_t i = 0; i < 9; i++) {
            lcd_data(no_device[i]);
        }
        no_device_status = 1;
        continue;
    }

    int16_t val = reading;
    double temperature_val = val / 16.0;
    uint8_t negative_sign = (temperature_val < 0) ? 1 : 0;

    if (negative_sign) temperature_val = -temperature_val;

    uint16_t integer = temperature_val;
    uint16_t decimal = (temperature_val - integer) * 10000;

    binary_to_bcd(integer);

    lcd_clear();
    if (negative_sign) {
        lcd_data('-');
    } else {
        lcd_data('+');
    }

    if (hundreds != 0) display_number(hundreds);
    if (tens != 0 || hundreds != 0) display_number(tens);
    display_number(units);
    lcd_data('.');

    binary_to_bcd(decimal);
    display_number(thousands);
    display_number(hundreds);
    display_number(tens);
    display_number(units);
    lcd_data(0b11011111);
    lcd_data('C');

    _delay_ms(500);
}
}

```